The Lattice Schwinger Model: A Complete Theoretical Specification for the "String Breaker" Quantum Simulation Project

Physics Lead, Bitcamp 2026

April 10, 2026

Abstract

This document is the complete theoretical specification for our quantum-computing project: a real-time simulation of the (1+1)-dimensional Schwinger model, running on IBM/IonQ-compatible Qiskit infrastructure, with an interactive puzzle-game frontend. It is written to be handed directly to teammates who are strong programmers but not necessarily trained in quantum field theory or lattice gauge theory. Nothing is hand-waved. The document starts from the continuum Lagrangian, derives the Hamiltonian by canonical quantization, discretizes onto a spatial lattice using Kogut–Susskind staggered fermions, applies the Jordan–Wigner transformation to map fermions to qubits, eliminates the gauge field via Gauss's law, and arrives at a final spin Hamiltonian that can be entered directly into Qiskit's `SparsePauliOp`. Sections on observables, initial state preparation, Trotterized time evolution, the variational ansatz, and the game-action-to-quantum-operation mapping close out the document. A worked N=2 example and an explicit implementation checklist appear at the end.

Contents

1 Purpose, Audience, and How to Read This Document **1**
- 1.1 What this project is . . . 1
- 1.2 Who this document is for . . . 2
- 1.3 How to read it . . . 2
- 1.4 What this document does not cover . . . 2

2 Notation and Conventions **2**
- 2.1 Units . . . 3
- 2.2 Spacetime indices and metric . . . 3
- 2.3 Pauli matrices . . . 3
- 2.4 Lattice conventions . . . 4
- 2.5 Summary table . . . 4

3 The Continuum Schwinger Model **4**
- 3.1 Ingredients of the Lagrangian . . . 4
- 3.2 Choosing the gamma matrices . . . 5
- 3.3 Unpacking the field-strength kinetic term . . . 6
- 3.4 Unpacking the fermion term . . . 6

4 Canonical Quantization in the Continuum **7**
4.1 Canonical momenta . . . 7
4.2 Gauss's law as the equation of motion for A_0 . . . 8
4.3 Temporal gauge fixing . . . 8
4.4 The Legendre transform . . . 9

5 Lattice Discretization: Kogut–Susskind Staggered Fermions **9**
5.1 The fermion doubling problem . . . 9
5.2 The Kogut–Susskind trick . . . 10
5.3 The Kogut–Susskind Hamiltonian . . . 10
5.4 The axial gauge . . . 11

6 The Jordan–Wigner Transformation **12**
6.1 Statement of the JW transformation . . . 12
6.2 Number operator at a single site . . . 12
6.3 Hopping term: the full derivation . . . 13
6.3.1 Rewriting in terms of X and Y . . . 13
6.3.2 Assembling the hopping Hamiltonian . . . 14
6.4 Mass term . . . 14
6.5 Vacuum verification . . . 15

7 Gauss's Law and Elimination of the Gauge Field **15**
7.1 Lattice Gauss's law . . . 15
7.2 The charge operator in Pauli form . . . 15
7.3 Solving the Gauss's law recurrence . . . 16
7.4 Interpretation of the background field ϵ_0 . . . 16

8 The Final Qubit Hamiltonian **17**
8.1 Term counting . . . 17
8.2 Expanding the electric energy . . . 18
8.3 Sanity check: the $N = 2$ case . . . 18

9 Observables for the Frontend **19**
9.1 Electric field on each link . . . 19
9.2 Particle number and charge at each site . . . 19
9.3 Total energy . . . 20
9.4 Chiral condensate (bonus observable) . . . 20

10 Initial State Preparation **20**
10.1 The bare vacuum . . . 20
10.2 A single fermion-antifermion pair . . . 20
10.3 Why "drag" is a re-initialization, not a gate . . . 21

11 Time Evolution and Trotterization **21**
11.1 First-order Trotter . . . 21
11.2 Second-order Trotter (Strang splitting) . . . 22
11.3 Qiskit implementation . . . 22
11.4 Step size and accuracy . . . 22

12 Variational Quantum Eigensolver: Level 4 **23**
12.1 VQE in three sentences . . . 23
12.2 The hardware-efficient ansatz . . . 23
12.3 Exposing the right parameters to the player . . . 23
12.4 The pitch line . . . 24

13 Exact Diagonalization: The Ground Truth **24**

14 Recommended Parameter Values **24**

15 Implementation Checklist and Game-Action Mapping **25**
15.1 The `SchwingerGameState` class . . . 25
15.2 Building the Hamiltonian . . . 27
15.3 Game action → quantum operation table . . . 28
15.4 Caching for responsiveness . . . 29
15.5 The one dramatic hardware run . . . 29
15.6 Things the coders should **not** do . . . 29
15.7 The recommended build order . . . 30

16 References and Further Reading **30**

A Appendix: Anticommutation of Jordan–Wigner Operators **31**

B Appendix: A Complete Pauli Decomposition for $N = 4$ **31**

---

1 Purpose, Audience, and How to Read This Document

1.1 What this project is

We are going to simulate the Schwinger model — quantum electrodynamics in one space and one time dimension — on a quantum computer. This is the simplest gauge theory that exhibits *confinement*, the phenomenon by which charged particles cannot be separated arbitrarily far because the electric field between them stores an ever-growing amount of energy. At some critical separation, it becomes energetically favorable for the vacuum to spontaneously produce a particle–antiparticle pair that "breaks" the field string: the original charges are now screened by their new partners, and the energy drops. This process, called *string breaking*, is the heart of both confinement physics and our game.

The frontend of our project is a puzzle game called "String Breaker." The player is shown a row of lattice sites with charged particles on them, a colored "field string" stretched between the charges, and a handful of sliders (coupling strength, mass, background field, time evolution). The player's job is to manipulate these sliders to break the string, to minimize energy, or to replicate target configurations. Under the hood, every slider motion corresponds to a real quantum computation: the wavefunction is updated, the Hamiltonian is remeasured, and the visible field and particles redraw themselves. Level 4, secretly, is a variational quantum eigensolver (VQE) in which the player's sliders *are* the parameters of a quantum circuit, and the player is literally doing the job of the classical optimizer in a hybrid quantum–classical algorithm.

1.2 Who this document is for

You — the reader — are a strong programmer and a competent mathematician, but possibly new to quantum field theory, gauge theory, and lattice discretizations. This document assumes you know:

- Linear algebra (eigenvalues, tensor products, Hermitian conjugation).
- Basic quantum mechanics (states, operators, expectation values, time evolution under a Hamiltonian).
- The Pauli matrices and elementary quantum gate concepts (CNOT, R_y, R_z).
- Python and some exposure to Qiskit, or willingness to read Qiskit docs as you go.

It does *not* assume you know what a Dirac spinor is, what a gauge transformation is, what canonical quantization is, or what the Jordan–Wigner transformation is. All of these are introduced in the text with enough care that you can understand the final formulas without faith.

1.3 How to read it

If you are in a hurry, skip to Section 8, which states the final qubit Hamiltonian you need to implement, and then Section 15, which is the literal checklist. Everything else is the derivation, and you should read it at least once so you can recognize what's correct when the simulator starts returning numbers. If the numbers look wrong at 3am on Saturday, the difference between "I understand the derivation" and "I copied formulas" is the difference between finding the bug and not.

If you want to be truly rigorous, read sections 3 through 7 linearly. They build from the continuum Lagrangian to the final spin Hamiltonian step by step, and every non-trivial claim is either derived in the text or given a sanity-check computation.

1.4 What this document does not cover

This document is about the physics and the quantum circuits. It does *not* cover: the frontend React components, the FastAPI server, the authentication to IonQ's cloud, or the game-level design. Those are separate documents. What this document *does* provide that the other documents depend on is the precise specification of the `SchwingerGameState` Python class (Section 15), which is the bridge between the physics backend and the rest of the stack.

2 Notation and Conventions

Before we begin, we pin down every notational choice. Every later section assumes these conventions, and inconsistency between sections is the most common source of sign errors. **If a sign in the code disagrees with a sign in this document, the document is the reference, not the code.**

2.1 Units

We use *natural units* throughout: $\hbar = c = 1$. A consequence is that time and length have the same units, and energy has units of inverse length. In our lattice simulation, we set the lattice spacing a to equal 1 by convention, so all energies are measured in units of $1/a$. If you want to re-introduce

a explicitly, every formula in this document still holds — the factor of a is kept explicit in the Hamiltonian.

2.2 Spacetime indices and metric

The spacetime dimension is $1+1$: one temporal dimension, indexed by $\mu = 0$, and one spatial dimension, indexed by $\mu = 1$. Greek indices μ, ν, ρ always run over $\{0, 1\}$. We use the metric signature

$$\eta_{\mu\nu} = \eta^{\mu\nu} = \begin{pmatrix} +1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{1}$$

This is the "mostly minus" convention (standard in particle physics). Indices are raised and lowered with η: $A^\mu = \eta^{\mu\nu} A_\nu$. Repeated indices are summed (Einstein convention).

2.3 Pauli matrices

We use both the names $\sigma^x, \sigma^y, \sigma^z$ and the names X, Y, Z interchangeably:

$$X = \sigma^x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \sigma^y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \sigma^z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2}$$

Key algebraic facts we will use repeatedly:

$$X^2 = Y^2 = Z^2 = I, \quad XY = iZ, \quad YZ = iX, \quad ZX = iY, \quad \{X, Y\} = \{Y, Z\} = \{Z, X\} = 0. \tag{3}$$

Computational basis and Z eigenvalues:

$$Z\,|0\rangle = +\,|0\rangle\,, \qquad Z\,|1\rangle = -\,|1\rangle\,. \tag{4}$$

Raising and lowering operators:

$$\sigma^+ = |0\rangle\langle 1| = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad \sigma^- = |1\rangle\langle 0| = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}. \tag{5}$$

In terms of X and Y:

$$\sigma^+ = \tfrac{1}{2}(X + iY), \qquad \sigma^- = \tfrac{1}{2}(X - iY). \tag{6}$$

Important: in our convention, σ^+ takes $|1\rangle \to |0\rangle$, i.e., it *raises* the Z eigenvalue from -1 to $+1$. When we map this to fermion occupation numbers later (where $|0\rangle$ = empty and $|1\rangle$ = occupied), σ^+ will correspond to the fermion *annihilation* operator and σ^- to the fermion *creation* operator. This is the reverse of what you might expect from the symbols, and is the source of half of all Jordan–Wigner sign errors in the literature. Learn this now and you will thank yourself later.

2.4 Lattice conventions

The lattice has N sites, where N is always an even positive integer. Sites are labeled by an integer $n \in \{1, 2, \ldots, N\}$ — note: **1-indexed**, because the mass staggering $(-1)^n$ is cleanest when the first site is odd. When we translate to Python, which is 0-indexed, we will have to be vigilant about off-by-one sign flips. There is a bright red warning about this in Section 15.

A "link" is the edge between two adjacent sites. We label link n as the link between site n and site $n+1$, so links are indexed $n \in \{1, \ldots, N-1\}$. The spatial geometry is a finite open chain; boundary conditions are *open* (no wraparound from site N to site 1). Open boundary conditions make the Gauss's law analysis cleaner and let us expose the background electric field as a free parameter.

2.5 Summary table

Symbol	Meaning	Units
a	Lattice spacing	length (inverse energy)
N	Number of lattice sites (even)	dimensionless
n	Site index, $n = 1, \ldots, N$	dimensionless
m	Fermion bare mass	energy
g	Gauge coupling	$\sqrt{\text{energy/length}}$ in (1+1)D
ϵ_0	Background electric field (dimensionless)	dimensionless
θ	Topological angle, $\theta = 2\pi\epsilon_0$	dimensionless
ϕ_n	Single-component staggered fermion operator at site n	—
U_n	Compact gauge link variable on link n	dimensionless
L_n	Dimensionless electric field on link n	dimensionless
X_n, Y_n, Z_n	Pauli operators acting on qubit n	—
$\sigma_n^{\pm}$	Raising/lowering operators on qubit n	—
$\lvert 0\rangle, \lvert 1\rangle$	Single-qubit basis states	—
$\lvert \text{vac}\rangle$	Kogut–Susskind reference vacuum $\lvert 0101\ldots01\rangle$	—
Q_n	Charge operator at site n	dimensionless
H	The spin Hamiltonian we actually implement	energy

3 The Continuum Schwinger Model

We begin in the continuum. The Schwinger model is defined by the Lagrangian density

$$\mathcal{L} = -\frac{1}{4}F^{\mu\nu}F_{\mu\nu} + \bar{\psi}\left(i\gamma^\mu D_\mu - m\right)\psi. \tag{7}$$

This looks intimidating if it is the first time you have seen it, so let us unpack every symbol, verify every sign, and derive everything we will need.

3.1 Ingredients of the Lagrangian

The fermion field ψ.

$\psi(x,t)$ is a function of spacetime whose values are *two-component column vectors*: $\psi = \begin{pmatrix}\psi_u \\ \psi_l\end{pmatrix}$. The index on the components is called a "spinor index" and runs over $\{u, l\}$ ("upper" and "lower"). In (3+1)D, spinors have four components; in (1+1)D, two suffice. After quantization, the components of ψ become operators that create and destroy fermions.

The Dirac conjugate $\bar{\psi}$.

This is *not* just the Hermitian conjugate. It is defined as $\bar{\psi} \equiv \psi^\dagger\gamma^0$. The extra γ^0 is there to make the combination $\bar{\psi}\psi$ Lorentz-invariant. You do not need to understand why from first principles; you need to know that this is the definition, and it matters when we take Hermitian conjugates later.

The gauge field A_μ.

$A_\mu(x,t)$ is a 2-component (one time component and one space component) real-valued field. A_0 is the scalar potential and A_1 is the vector potential, to use the names familiar from classical electrodynamics. In our 2D world, the "vector" potential has only one component because there is only one spatial direction.

The field strength $F_{\mu\nu}$.
Defined as $F_{\mu\nu} = \partial_\mu A_\nu - \partial_\nu A_\mu$. It is antisymmetric, so in 2D it has only one independent component: $F_{01} = \partial_0 A_1 - \partial_1 A_0$. We will see below that this single component is the electric field.

The covariant derivative D_μ.
Defined as $D_\mu = \partial_\mu + igA_\mu$. It is what makes the theory *gauge-invariant*: if you change $\psi \to e^{i\alpha(x)}\psi$ and simultaneously $A_\mu \to A_\mu - (1/g)\partial_\mu \alpha$, the Lagrangian is unchanged. We will not need to use gauge invariance directly for derivations, but it is the reason the theory has the structure it does.

The Dirac matrices γ^μ.
Two 2×2 matrices satisfying the Clifford algebra $\{\gamma^\mu, \gamma^\nu\} = 2\eta^{\mu\nu} I$. Several conventions exist; we will pick one below.

The mass parameter m.
A real positive number with dimensions of energy. The limit $m = 0$ is the "massless Schwinger model," which is exactly solvable; $m > 0$ is the "massive Schwinger model," which is not exactly solvable but is the physically interesting regime for confinement and string breaking.

The coupling g.
The charge of the fermion, with dimensions that ensure gA_μ has dimensions of inverse length (so that D_μ has the same dimensions as ∂_μ). In (1+1)D this works out to g having dimensions of energy.

3.2 Choosing the gamma matrices

The gamma matrices are required to satisfy $\{\gamma^\mu, \gamma^\nu\} = 2\eta^{\mu\nu} I$, i.e.,

$$(\gamma^0)^2 = +I, \quad (\gamma^1)^2 = -I, \quad \gamma^0\gamma^1 + \gamma^1\gamma^0 = 0. \tag{8}$$

The smallest representation in 2D is via 2×2 matrices. We choose:

$$\gamma^0 = \sigma^z, \qquad \gamma^1 = i\sigma^y. \tag{9}$$

Verification. $(\gamma^0)^2 = (\sigma^z)^2 = I$, check. $(\gamma^1)^2 = (i\sigma^y)^2 = -(\sigma^y)^2 = -I$, check. And

$$\{\gamma^0, \gamma^1\} = \sigma^z(i\sigma^y) + (i\sigma^y)\sigma^z = i\{\sigma^z, \sigma^y\} = 0 \tag{10}$$

because σ^z and σ^y anticommute. All three Clifford conditions are satisfied.

A useful derived quantity:

$$\gamma^0\gamma^1 = \sigma^z(i\sigma^y) = i\sigma^z\sigma^y = i(-i\sigma^x) = \sigma^x, \tag{11}$$

where we used $\sigma^z\sigma^y = -i\sigma^x$ (which follows from $\sigma^y\sigma^z = i\sigma^x$ and anticommutation). So

$$\gamma^0\gamma^1 = \sigma^x. \tag{12}$$

3.3 Unpacking the field-strength kinetic term

We compute $-\frac{1}{4}F^{\mu\nu}F_{\mu\nu}$ explicitly. Because $F_{\mu\nu}$ is antisymmetric, the only nonzero components are F_{01} and $F_{10} = -F_{01}$. Therefore

$$F_{\mu\nu}F^{\mu\nu} = F_{01}F^{01} + F_{10}F^{10} = 2F_{01}F^{01}. \tag{13}$$

Now we raise the indices:

$$F^{01} = \eta^{0\alpha}\eta^{1\beta}F_{\alpha\beta} = \eta^{00}\eta^{11}F_{01} = (+1)(-1)F_{01} = -F_{01}. \tag{14}$$

Therefore

$$F_{\mu\nu}F^{\mu\nu} = 2F_{01}(-F_{01}) = -2(F_{01})^2, \tag{15}$$

and

$$-\frac{1}{4}F^{\mu\nu}F_{\mu\nu} = -\frac{1}{4}(-2)(F_{01})^2 = \frac{1}{2}(F_{01})^2. \tag{16}$$

It is traditional (and physically correct in 2D) to *define* the electric field as $E \equiv F_{01} = \partial_0 A_1 - \partial_1 A_0$. There is no magnetic field in (1+1)D; the field strength has only one independent component, and it is all electric. So our gauge kinetic term is simply

$$\boxed{-\frac{1}{4}F^{\mu\nu}F_{\mu\nu} = \frac{1}{2}E^2.} \tag{17}$$

Done. This is literally $\frac{1}{2}$(electric field)^2, the familiar electrostatic energy density, with no magnetic part to worry about.

3.4 Unpacking the fermion term

We now compute $\bar{\psi}(i\gamma^\mu D_\mu - m)\psi$ with our choice of gammas. Substituting $\bar{\psi} = \psi^\dagger\gamma^0$ and expanding:

$$\begin{aligned}
\bar{\psi}(i\gamma^\mu D_\mu - m)\psi &= \psi^\dagger\gamma^0(i\gamma^0 D_0 + i\gamma^1 D_1 - m)\psi && (18)\\
&= i\psi^\dagger(\gamma^0)^2 D_0\psi + i\psi^\dagger\gamma^0\gamma^1 D_1\psi - m\psi^\dagger\gamma^0\psi && (19)\\
&= i\psi^\dagger D_0\psi + i\psi^\dagger\sigma^x D_1\psi - m\psi^\dagger\sigma^z\psi, && (20)
\end{aligned}$$

where we used $(\gamma^0)^2 = I$, the product (12), and $\gamma^0 = \sigma^z$. Finally, expanding the covariant derivatives $D_\mu = \partial_\mu + igA_\mu$:

$$\begin{aligned}
\bar{\psi}(i\gamma^\mu D_\mu - m)\psi = {} & i\psi^\dagger\partial_0\psi - gA_0\,\psi^\dagger\psi \\
& + i\psi^\dagger\sigma^x\partial_1\psi - gA_1\,\psi^\dagger\sigma^x\psi \\
& - m\,\psi^\dagger\sigma^z\psi.
\end{aligned} \tag{21}$$

Combining with the gauge kinetic term, the full Lagrangian density is

$$\begin{aligned}
\mathcal{L} = {} & \frac{1}{2}(\partial_0 A_1 - \partial_1 A_0)^2 \\
& + i\psi^\dagger\partial_0\psi - gA_0\,\psi^\dagger\psi \\
& + i\psi^\dagger\sigma^x\partial_1\psi - gA_1\,\psi^\dagger\sigma^x\psi - m\,\psi^\dagger\sigma^z\psi.
\end{aligned} \tag{22}$$

Every term in the theory is now written out in terms of scalar fields $(\psi_u, \psi_l, A_0, A_1)$ and Pauli matrices acting on the spinor index. There is no more γ^μ machinery left.

4 Canonical Quantization in the Continuum

With the Lagrangian in hand, we pass to the Hamiltonian formalism via canonical quantization. The recipe has three steps: identify the canonical momenta, identify any constraints (fields with zero momentum), and compute the Hamiltonian density via the Legendre transform.

4.1 Canonical momenta

The canonical momentum conjugate to a field ϕ is $\pi_\phi = \partial\mathcal{L}/\partial(\partial_0\phi)$. Reading off from (22):

Fermion momentum. The term in $\mathcal{L}$ with a time derivative of ψ is $i\psi^\dagger\partial_0\psi$. So

$$\pi_\psi = \frac{\partial\mathcal{L}}{\partial(\partial_0\psi)} = i\psi^\dagger. \tag{23}$$

This tells us $\psi^\dagger$ plays the role of the conjugate momentum to ψ. After quantization, ψ and $\psi^\dagger$ obey the equal-time anticommutation relations

$$\{\psi_\alpha(x), \psi^\dagger_\beta(y)\} = \delta_{\alpha\beta}\delta(x-y), \qquad \{\psi_\alpha(x), \psi_\beta(y)\} = 0, \tag{24}$$

where α, β are spinor indices.

Spatial gauge-field momentum. The only term with a time derivative of A_1 is $\frac{1}{2}(\partial_0 A_1 - \partial_1 A_0)^2$. So

$$\pi_{A_1} = \frac{\partial\mathcal{L}}{\partial(\partial_0 A_1)} = \partial_0 A_1 - \partial_1 A_0 = F_{01} = E. \tag{25}$$

The canonical momentum of A_1 is the electric field E. This is the deep reason electric field is the "momentum" of the gauge field in the Hamiltonian formulation. After quantization, $[A_1(x), E(y)] = i\delta(x-y)$.

Temporal gauge-field momentum. No term in $\mathcal{L}$ contains $\partial_0 A_0$. Therefore

$$\pi_{A_0} = 0. \tag{26}$$

This is a *primary constraint*: it tells us that A_0 is *not a dynamical field*. It has no canonical momentum, no time evolution. It appears in the Lagrangian only as a Lagrange multiplier enforcing another condition. What condition? Gauss's law, which we derive next.

4.2 Gauss's law as the equation of motion for A_0

Because A_0 has no time derivative in the action, varying the action with respect to A_0 gives an algebraic constraint rather than a dynamical equation. The Euler–Lagrange equation for A_0 is

$$\frac{\partial\mathcal{L}}{\partial A_0} - \partial_\mu \frac{\partial\mathcal{L}}{\partial(\partial_\mu A_0)} = 0. \tag{27}$$

From (22), the only term containing A_0 without a derivative is $-gA_0\,\psi^\dagger\psi$, giving $\partial\mathcal{L}/\partial A_0 = -g\psi^\dagger\psi$. The only term containing a derivative of A_0 is the $(\partial_1 A_0)$ inside the square, giving

$$\frac{\partial\mathcal{L}}{\partial(\partial_1 A_0)} = -(\partial_0 A_1 - \partial_1 A_0) = -E. \tag{28}$$

There is no $\partial_0 A_0$ term, so $\partial\mathcal{L}/\partial(\partial_0 A_0) = 0$. The EL equation becomes

$$-g\psi^\dagger\psi - \partial_1(-E) = 0 \quad \Longrightarrow \quad \partial_1 E = g\psi^\dagger\psi. \tag{29}$$

This is **Gauss's law** in (1+1)D:

$$\boxed{\partial_1 E(x,t) = g\,\psi^\dagger\psi(x,t).} \tag{30}$$

In words: the spatial derivative of the electric field equals the charge density. If you have only ever seen Gauss's law as $\nabla\cdot\vec{E} = \rho$, this is the same equation with the vector calculus simplified to 1D. After we quantize, this equation becomes a *constraint on physical states*: only state vectors $|\Psi\rangle$ satisfying $(\partial_1\hat{E} - g\hat{\psi}^\dagger\hat{\psi})\,|\Psi\rangle = 0$ at every point are considered "physical." States that violate Gauss's law are unphysical and are projected out.

This is the single most important fact of the entire derivation. Gauss's law is the reason we can eliminate the gauge field entirely. In higher dimensions gauge fields have their own propagating degrees of freedom (photons, gluons), but in (1+1)D they do not: Gauss's law determines the electric field everywhere from the charges. We will exploit this ruthlessly in Section 7.

4.3 Temporal gauge fixing

The Lagrangian has a gauge symmetry: the physics is unchanged under

$$\psi(x) \to e^{i\alpha(x)}\psi(x), \qquad A_\mu(x) \to A_\mu(x) - \frac{1}{g}\partial_\mu\alpha(x). \tag{31}$$

We use this freedom to pick a gauge. Our choice is the *temporal gauge*:

$$A_0(x,t) \equiv 0. \tag{32}$$

(This is always achievable by choosing α such that $\partial_0\alpha = gA_0$.) In this gauge, the Lagrangian simplifies:

$$\mathcal{L} = \tfrac{1}{2}(\partial_0 A_1)^2 + i\psi^\dagger\partial_0\psi + i\psi^\dagger\sigma^x\partial_1\psi - gA_1\psi^\dagger\sigma^x\psi - m\psi^\dagger\sigma^z\psi. \tag{33}$$

Gauss's law is not automatic in this gauge; it must still be imposed by hand as a constraint on the physical Hilbert space.

4.4 The Legendre transform

The Hamiltonian density is

$$\mathcal{H} = \pi_\psi\partial_0\psi + \pi_{A_1}\partial_0 A_1 - \mathcal{L}. \tag{34}$$

Substituting $\pi_\psi = i\psi^\dagger$ and $\pi_{A_1} = E = \partial_0 A_1$ (in temporal gauge):

$$\begin{aligned}\mathcal{H} = {} & i\psi^\dagger\partial_0\psi + (\partial_0 A_1)^2 - \tfrac{1}{2}(\partial_0 A_1)^2 \\ & - i\psi^\dagger\partial_0\psi - i\psi^\dagger\sigma^x\partial_1\psi + gA_1\psi^\dagger\sigma^x\psi + m\psi^\dagger\sigma^z\psi.\end{aligned} \tag{35}$$

The $\partial_0\psi$ terms cancel (as they always do for first-order-in-time fields). The $(\partial_0 A_1)^2$ terms combine to give $\frac{1}{2}(\partial_0 A_1)^2 = \frac{1}{2}E^2$. The remaining fermionic terms combine into a covariant-derivative form:

$$-i\psi^\dagger\sigma^x\partial_1\psi + gA_1\psi^\dagger\sigma^x\psi = \psi^\dagger\sigma^x(-i\partial_1 + gA_1)\psi = -i\psi^\dagger\sigma^x D_1\psi, \tag{36}$$

where $D_1 = \partial_1 + igA_1$. (The rearrangement works because $-i\partial_1 + gA_1 = -i(\partial_1 + igA_1) = -iD_1$.)

The final Hamiltonian density is

$$\boxed{\mathcal{H} = \tfrac{1}{2}E^2 - i\psi^\dagger\sigma^x D_1\psi + m\psi^\dagger\sigma^z\psi.} \tag{37}$$

Interpretation. The three terms are:

1. $\frac{1}{2}E^2$: energy stored in the electric field (classical Coulomb energy density in 1D).
2. $-i\psi^\dagger\sigma^x D_1\psi$: fermion kinetic energy, including coupling to the gauge field. This term will become the hopping term on the lattice.
3. $m\psi^\dagger\sigma^z\psi$: fermion mass. The σ^z distinguishes the upper and lower components of the Dirac spinor (which will become even and odd lattice sites).

Plus Gauss's law (30) as a constraint. This is the continuum Schwinger model, ready to discretize.

5 Lattice Discretization: Kogut–Susskind Staggered Fermions

We now put the continuum theory on a spatial lattice of N sites with lattice spacing a. Time remains continuous, so we are building a *Hamiltonian lattice* formulation (as opposed to a path-integral lattice formulation, in which time would also be discretized). This is the right choice for quantum simulation because the output of a quantum computer is the wavefunction at a given time, not a path integral.

5.1 The fermion doubling problem

The first thing you might try is to discretize spatial derivatives with a symmetric finite difference:

$$\partial_1\psi(x) \approx \frac{\psi(x+a)-\psi(x-a)}{2a}. \tag{38}$$

This is the familiar centered difference from numerical analysis, and it has second-order accuracy. On the lattice, the derivative term in the Dirac operator becomes

$$-i\psi^\dagger\sigma^x\partial_1\psi \to \frac{-i}{2a}\sum_n\left[\psi_n^\dagger\sigma^x\psi_{n+1}-\psi_n^\dagger\sigma^x\psi_{n-1}\right], \tag{39}$$

where ψ_n is the spinor at site n.

This looks fine. It is not fine. The problem becomes visible when you compute the dispersion relation of the resulting lattice Dirac operator. In momentum space, $\psi_n = e^{ikna}\psi_k$, and the centered difference gives

$$\text{(kinetic operator in momentum space)} \propto \sin(ka). \tag{40}$$

This function has a zero not only at $k=0$ (as the continuum did) but also at $k=\pi/a$, the *edge of the Brillouin zone*. An extra zero means an extra massless fermion mode where you didn't want one. In (1+1)D, this doubles the number of fermion species: you asked for one and you got two. In (3+1)D, the doubling is $2^3=8$.

This is the fermion doubling problem, and it is a fundamental obstruction to naive lattice fermions. The Nielsen–Ninomiya theorem formalizes it: you cannot write down a local, Hermitian, translation-invariant, chirally-symmetric lattice fermion action without doublers. Something has to give.

5.2 The Kogut–Susskind trick

Kogut and Susskind (1975) found a clever workaround that keeps locality and Hermiticity at the price of chiral symmetry (partially). The idea: *don't put the whole spinor on each site.* Instead, distribute the components of the Dirac spinor across neighboring sites.

In (1+1)D, the Dirac spinor has two components, $\psi = (\psi_u, \psi_l)^T$. The staggered rearrangement is to place ψ_u on odd sites of the lattice and ψ_l on even sites. From the perspective of a single chain, we now have one fermionic degree of freedom per site (call it ϕ_n) instead of two. Because the number of fermion modes per unit length has been halved, the effective number of continuum species has also been halved: $2^1/2 = 1$, which is what we wanted.

In higher dimensions the counting is more subtle (you get $2^d/2^d = 1$ *tastes* times some number per site, and you have to decouple them afterwards), but in (1+1)D the staggered construction gives exactly one continuum fermion species. This is the simplest case where the Kogut–Susskind approach works essentially perfectly, which is another reason the Schwinger model is the natural playground for learning lattice gauge theory.

5.3 The Kogut–Susskind Hamiltonian

The staggered construction, combined with a naive discretization of the gauge-fermion coupling and the electric energy, gives the Kogut–Susskind Hamiltonian. The derivation from the continuum Hamiltonian (37) involves a specific redefinition of fermion fields that absorbs some factors of i into the staggered phase, and the details are in the original 1975 paper and in the review [**?**]. We skip those details — they are lengthy and concern the justification of the form rather than its content — and simply state the result:

$$H_{\mathrm{KS}} = -\frac{1}{2a}\sum_{n=1}^{N-1}\Big[\phi_n^\dagger U_n \phi_{n+1} + \text{h.c.}\Big] + m\sum_{n=1}^{N}(-1)^{n+1}\phi_n^\dagger\phi_n + \frac{ag^2}{2}\sum_{n=1}^{N-1}L_n^2. \tag{41}$$

Here the ingredients are:

ϕ_n is a *single-component* fermionic annihilation operator at site n, satisfying the canonical anti-commutation relations $\{\phi_n, \phi_m^\dagger\} = \delta_{nm}$ and $\{\phi_n, \phi_m\} = 0$. The single-component-ness is the result of staggering: the spinor index has been absorbed into the site-parity index.

$U_n = e^{i\theta_n}$
is a compact *link variable* on link n, where $\theta_n = agA_{1,n}$ is the dimensionless gauge phase built from the continuum gauge field A_1 averaged over the link. U_n is a unitary operator with eigenvalues on the unit circle. It transforms under gauge transformations as $U_n \to e^{i\alpha_n}U_n e^{-i\alpha_{n+1}}$.

L_n is the dimensionless electric field on link n, conjugate to θ_n in the sense $[\theta_n, L_m] = i\delta_{nm}$. In the compact formulation L_n has integer eigenvalues (because θ_n is periodic), but for our purposes we will not need to impose that: we will eliminate L_n in favor of charges via Gauss's law, and only the charges will be meaningful.

The sign $(-1)^{n+1}$
in the mass term. This is a choice of convention: with this sign, the $n = 1$ site contributes $+m\phi_1^\dagger\phi_1$ and the $n = 2$ site contributes $-m\phi_2^\dagger\phi_2$. Equivalently, placing a fermion on an odd site costs mass energy, while removing one from an even site gains mass energy. We will see in Section 6 that this sign ensures the Kogut–Susskind vacuum is $|0101\ldots01\rangle$ (first qubit empty) rather than $|1010\ldots\rangle$.

Interpreting each term.

- The *hopping term* $-\frac{1}{2a}\phi_n^\dagger U_n \phi_{n+1}$ destroys a fermion at site $n+1$ and creates one at site n (with a gauge link in between to keep the operation gauge-invariant). Plus its Hermitian conjugate, which does the reverse. This is the lattice version of the fermion kinetic term $-i\psi^\dagger \sigma^x D_1 \psi$.
- The *mass term* $m(-1)^{n+1}\phi_n^\dagger \phi_n$ is a staggered on-site energy. Because of the alternating sign, odd sites want to be empty and even sites want to be occupied (for positive m), which is exactly the structure of the Dirac sea.
- The *electric energy* $(ag^2/2)L_n^2$ is the lattice version of $\frac{1}{2}E^2$. You can check the units: L_n is dimensionless, g^2 has units of $1/a^2$ in (1+1)D, so ag^2 has units of $1/a$ = energy, as expected for a Hamiltonian.

5.4 The axial gauge

We have already used the temporal gauge $A_0 = 0$. On a finite open chain in (1+1)D, we can use the *residual* time-independent gauge freedom to set all the link variables to the identity:

$$U_n \equiv 1 \quad \text{for all } n. \tag{42}$$

This is called the *axial gauge*. Setting every U_n to 1 may sound dramatic — are we not throwing away all the gauge field? — but it is not. The *configuration* variables θ_n are zero, yes, but the *conjugate momenta* L_n are not. The electric fields survive. What we have done is chosen a gauge in which the gauge field is not encoded in the link variables, and it is instead entirely encoded in the electric-field operators L_n.

In this gauge the hopping term becomes

$$-\frac{1}{2a}\sum_n \left[\phi_n^\dagger \phi_{n+1} + \phi_{n+1}^\dagger \phi_n\right], \tag{43}$$

and the Hamiltonian (41) reads

$$H = -\frac{1}{2a}\sum_{n=1}^{N-1}\left[\phi_n^\dagger \phi_{n+1} + \text{h.c.}\right] + m\sum_{n=1}^{N}(-1)^{n+1}\phi_n^\dagger \phi_n + \frac{ag^2}{2}\sum_{n=1}^{N-1} L_n^2. \tag{44}$$

We still have the fermionic operators ϕ_n and the electric operators L_n. Next, Jordan–Wigner turns the fermions into qubits; then Gauss's law eliminates L_n.

6 The Jordan–Wigner Transformation

The Jordan–Wigner (JW) transformation is a map from 1D lattice fermions to qubits (spin-1/2 sites). It is exact: every fermionic operator is mapped to a specific Pauli-operator combination, and the anticommutation relations of the fermions are reproduced by the commutation relations of the Paulis together with a non-local "string."

6.1 Statement of the JW transformation

We assign one qubit to each lattice site n. The mapping is:

$$\phi_n = \left(\prod_{\ell=1}^{n-1}(-Z_\ell)\right)\sigma_n^+, \tag{45}$$

$$\phi_n^\dagger = \left(\prod_{\ell=1}^{n-1}(-Z_\ell)\right)\sigma_n^-. \tag{46}$$

The product $\prod_{\ell=1}^{n-1}(-Z_\ell)$ is the *Jordan–Wigner string*. For $n = 1$, the product is empty and equals 1; for $n = 2$, it is $(-Z_1)$; and so on. The $\sigma_n^\pm$ acts on qubit n.

Why the signs work. Recall our convention: σ^+ takes $|1\rangle \to |0\rangle$ and σ^- takes $|0\rangle \to |1\rangle$. In the fermion-occupation picture where $|0\rangle$ = empty and $|1\rangle$ = occupied, σ^+ therefore *annihilates* a fermion and σ^- *creates* one. So ϕ_n (annihilation operator) contains σ_n^+, and $\phi_n^\dagger$ (creation operator) contains σ_n^-. This matches (45) and (46).

Why the string is needed. At two different sites $m \neq n$, the raw Pauli operators $\sigma_m^\pm$ and $\sigma_n^\pm$ *commute* (because they act on different qubits), whereas the fermionic operators $\phi_m^\dagger$ and $\phi_n^\dagger$ must *anticommute*. The JW string is precisely what converts commutation into anticommutation: each Z_ℓ anticommutes with the $\sigma^\pm$ at the same site, and the product of all the Z's between sites m and n introduces the correct sign when ϕ_m and ϕ_n are swapped. We prove this explicitly in Appendix A.

6.2 Number operator at a single site

The on-site fermion number is $\phi_n^\dagger\phi_n$. Because both operators act at the same site, the JW strings are identical and cancel (they square to the identity, since each $(-Z_\ell)$ squares to I). So

$$\phi_n^\dagger\phi_n = \sigma_n^-\sigma_n^+. \tag{47}$$

Now we compute $\sigma^-\sigma^+$ directly. By definition, $\sigma^+ = |0\rangle\langle 1|$ and $\sigma^- = |1\rangle\langle 0|$, so

$$\sigma^-\sigma^+ = |1\rangle\langle 0|\,|0\rangle\langle 1| = |1\rangle\langle 1|\,. \tag{48}$$

This is the projector onto $|1\rangle$, which in Pauli form is $(I - Z)/2$. Therefore

$$\boxed{\phi_n^\dagger\phi_n = \frac{I - Z_n}{2}.} \tag{49}$$

This operator has eigenvalue 0 on $|0\rangle$ (empty) and 1 on $|1\rangle$ (occupied), as a fermion number operator must.

6.3 Hopping term: the full derivation

We compute $\phi_n^\dagger\phi_{n+1}$ for adjacent sites. Substituting (45) and (46):

$$\phi_n^\dagger\phi_{n+1} = \left(\prod_{\ell=1}^{n-1}(-Z_\ell)\right)\sigma_n^-\left(\prod_{\ell=1}^{n}(-Z_\ell)\right)\sigma_{n+1}^+. \tag{50}$$

The two JW strings overlap on sites $1, \ldots, n-1$. Since each $(-Z_\ell)$ squares to I (because $Z_\ell^2 = I$ and $(-1)^2 = 1$), the overlap simply cancels. The second string has one extra factor, namely $(-Z_n)$, which survives:

$$\phi_n^\dagger \phi_{n+1} = \sigma_n^- \, (-Z_n) \, \sigma_{n+1}^+ = -\sigma_n^- Z_n \, \sigma_{n+1}^+. \tag{51}$$

(Here σ_{n+1}^+ commutes past Z_n because they act on different qubits.)

Now we simplify $\sigma_n^- Z_n$. On-site:

- Acting on $|0\rangle$: $\sigma^- Z \, |0\rangle = \sigma^- \cdot (+1) \, |0\rangle = \sigma^- \, |0\rangle = |1\rangle$. And $\sigma^- \, |0\rangle = |1\rangle$. So both give the same result: $\sigma^- Z \, |0\rangle = \sigma^- \, |0\rangle$.
- Acting on $|1\rangle$: $\sigma^- Z \, |1\rangle = \sigma^- \cdot (-1) \, |1\rangle = -\sigma^- \, |1\rangle = 0$. And $\sigma^- \, |1\rangle = 0$. So both are zero.

Therefore $\sigma^- Z = \sigma^-$ as an operator identity. Plugging back,

$$\phi_n^\dagger \phi_{n+1} = -\sigma_n^- \sigma_{n+1}^+. \tag{52}$$

And taking the Hermitian conjugate:

$$\phi_{n+1}^\dagger \phi_n = \left(\phi_n^\dagger \phi_{n+1}\right)^\dagger = -\left(\sigma_n^- \sigma_{n+1}^+\right)^\dagger = -\sigma_{n+1}^- \sigma_n^+ = -\sigma_n^+ \sigma_{n+1}^-. \tag{53}$$

(The last step uses the fact that operators on different qubits commute.)

So the hopping piece is

$$\phi_n^\dagger \phi_{n+1} + \phi_{n+1}^\dagger \phi_n = -\left[\sigma_n^- \sigma_{n+1}^+ + \sigma_n^+ \sigma_{n+1}^-\right]. \tag{54}$$

6.3.1 Rewriting in terms of X and Y

Using $\sigma^+ = (X + iY)/2$ and $\sigma^- = (X - iY)/2$:

$$\begin{aligned} \sigma_n^- \sigma_{n+1}^+ &= \tfrac{1}{4}(X_n - iY_n)(X_{n+1} + iY_{n+1}) \\ &= \tfrac{1}{4}\left[X_n X_{n+1} + iX_n Y_{n+1} - iY_n X_{n+1} + Y_n Y_{n+1}\right], \end{aligned} \tag{55}$$

$$\begin{aligned} \sigma_n^+ \sigma_{n+1}^- &= \tfrac{1}{4}(X_n + iY_n)(X_{n+1} - iY_{n+1}) \\ &= \tfrac{1}{4}\left[X_n X_{n+1} - iX_n Y_{n+1} + iY_n X_{n+1} + Y_n Y_{n+1}\right]. \end{aligned} \tag{56}$$

The imaginary cross-terms ($X_n Y_{n+1}$ and $Y_n X_{n+1}$) cancel when we add these two expressions, and the diagonal terms ($X_n X_{n+1}$ and $Y_n Y_{n+1}$) double:

$$\sigma_n^- \sigma_{n+1}^+ + \sigma_n^+ \sigma_{n+1}^- = \frac{1}{2}\left[X_n X_{n+1} + Y_n Y_{n+1}\right]. \tag{57}$$

6.3.2 Assembling the hopping Hamiltonian

From (44), the hopping piece of the Hamiltonian is

$$H_{\text{hop}} = -\frac{1}{2a} \sum_{n=1}^{N-1} \left[\phi_n^\dagger \phi_{n+1} + \phi_{n+1}^\dagger \phi_n\right] \tag{58}$$

$$= -\frac{1}{2a} \sum_{n=1}^{N-1} \left(-\tfrac{1}{2}[X_n X_{n+1} + Y_n Y_{n+1}]\right) \tag{59}$$

$$= \frac{1}{4a} \sum_{n=1}^{N-1} \left[X_n X_{n+1} + Y_n Y_{n+1}\right]. \tag{60}$$

This is the celebrated $XX + YY$ (or "XY model") form. Every fermion hopping term in our problem has become a nearest-neighbor spin-spin coupling on the qubit chain.

6.4 Mass term

From (44) and the number operator (49):

$$H_{\text{mass}} = m \sum_{n=1}^{N} (-1)^{n+1} \phi_n^\dagger \phi_n \tag{61}$$

$$= m \sum_{n=1}^{N} (-1)^{n+1} \cdot \frac{I - Z_n}{2} \tag{62}$$

$$= \frac{m}{2} \underbrace{\sum_{n=1}^{N} (-1)^{n+1}}_{=0 \text{ for even } N} - \frac{m}{2} \sum_{n=1}^{N} (-1)^{n+1} Z_n. \tag{63}$$

The first sum vanishes because N is even: the $+1$'s and -1's pair up. (If N were odd, we would have a constant offset, which is physically irrelevant but ugly. Keep N even.) The second sum survives, and using $(-1)^{n+1} = -(-1)^n$:

$$\boxed{H_{\text{mass}} = \frac{m}{2} \sum_{n=1}^{N} (-1)^n \, Z_n.} \tag{64}$$

Note the sign flip: we started with $(-1)^{n+1}\phi_n^\dagger \phi_n$ in the Hamiltonian, and after dropping the constant we end up with $(-1)^n Z_n$. The two signs are correlated; this sign flip is the reason our vacuum will turn out to be $|0101\ldots\rangle$ rather than the other one.

6.5 Vacuum verification

Let us check that the state $|\text{vac}\rangle = |0101\ldots01\rangle$ (with site 1 in $|0\rangle$, site 2 in $|1\rangle$, alternating) minimizes the mass energy among product states.

In this state, $Z_n |\text{vac}\rangle = (-1)^{n+1} |\text{vac}\rangle$: site 1 is $|0\rangle$ with $Z = +1$ (that is $(-1)^2 = +1$ when $n = 1$, good), site 2 is $|1\rangle$ with $Z = -1$ (that is $(-1)^3 = -1$ when $n = 2$, good), etc. So

$$\langle \text{vac} | H_{\text{mass}} | \text{vac} \rangle = \frac{m}{2} \sum_{n=1}^{N} (-1)^n (-1)^{n+1} = \frac{m}{2} \sum_{n=1}^{N} (-1)^{2n+1} = -\frac{mN}{2}. \tag{65}$$

For any other product state $|s_1 s_2 \ldots s_N\rangle$ with $s_n \in \{0, 1\}$, we have $\langle Z_n \rangle = 1 - 2s_n \in \{+1, -1\}$, and the mass energy is

$$\frac{m}{2} \sum_n (-1)^n (1 - 2s_n). \tag{66}$$

Minimizing site by site: at odd n (where $(-1)^n = -1$), we want $-(1-2s_n)$ minimized, i.e., $(1-2s_n)$ maximized, i.e., $s_n = 0$. At even n (where $(-1)^n = +1$), we want $(1 - 2s_n)$ minimized, i.e., $s_n = 1$. So the minimizer is exactly $|0101\ldots\rangle$, achieving $-mN/2$. ✓

The true ground state of the full H (not just H_{mass}) is *not* a product state, because the hopping term H_{hop} mixes the vacuum with particle-antiparticle pair states. But in the limit $m \gg 1/a$ (heavy fermion, weak hopping), the true ground state is close to $|\text{vac}\rangle$. For the game we always initialize to $|\text{vac}\rangle$ and let the evolution (or the VQE ansatz) take over from there.

7 Gauss's Law and Elimination of the Gauge Field

We have one term left to convert to Pauli operators: the electric energy $(ag^2/2)L_n^2$. The L_n are not fermions, so Jordan–Wigner does not apply. Instead we use Gauss's law to *solve for* L_n in terms of the charges, which are already expressible in terms of Z_n.

7.1 Lattice Gauss's law

The continuum law $\partial_1 E = g\psi^\dagger\psi$ discretizes to a difference equation. On our lattice, E lives on links and $\psi^\dagger\psi$ lives on sites, so the discretized derivative $(L_n - L_{n-1})/a$ at site n equals the charge density Q_n/a at site n (with an appropriate rescaling by g absorbed into the definition of dimensionless charge). The result is

$$L_n - L_{n-1} = Q_n, \qquad n = 1, 2, \ldots, N, \tag{67}$$

with the boundary convention $L_0 \equiv \epsilon_0$ (a *background* electric field to the left of the first site) and L_N determined implicitly by the charges.

7.2 The charge operator in Pauli form

The *staggered* charge operator subtracts off the Kogut–Susskind vacuum occupation so that $\langle \text{vac}|Q_n|\text{vac}\rangle = 0$ on every site:

$$Q_n = \phi_n^\dagger \phi_n - \underbrace{\frac{1+(-1)^n}{2}}_{\text{vacuum occupation at site } n}. \tag{68}$$

The subtraction $(1+(-1)^n)/2$ equals 0 for odd n and 1 for even n, matching the vacuum $|0101\ldots01\rangle$ which has odd sites empty and even sites occupied. Substituting the Pauli form of the number operator (49):

$$Q_n = \frac{I - Z_n}{2} - \frac{1+(-1)^n}{2} \tag{69}$$

$$= \frac{1 - Z_n - 1 - (-1)^n}{2} \tag{70}$$

$$= -\frac{Z_n + (-1)^n}{2}. \tag{71}$$

Sanity checks. In the vacuum, $Z_n = (-1)^{n+1}$, so $Z_n + (-1)^n = (-1)^{n+1} + (-1)^n = 0$, giving $Q_n = 0$. ✓

Flipping an odd-site qubit from $|0\rangle$ to $|1\rangle$ (creating a fermion at odd site n) changes Z_n from $+1$ to -1. Then $Z_n + (-1)^n = -1 + (-1) = -2$, and $Q_n = -(-2)/2 = +1$. So a fermion created on an odd site carries charge $+1$. ✓

Flipping an even-site qubit from $|1\rangle$ to $|0\rangle$ (removing a fermion from an even site — equivalently, creating an *antifermion* there) changes Z_n from -1 to $+1$. Then $Z_n + (-1)^n = +1 + (+1) = +2$, and $Q_n = -(+2)/2 = -1$. So an antifermion on an even site carries charge -1. ✓

In our convention, fermions are positively charged and antifermions are negatively charged. This is a *sign convention*, not a physical statement: the Schwinger model does not distinguish between "electron = negative" and "electron = positive" at the level of the Hamiltonian, only between the two species and their relative charges. In the game, we are free to color the fermions blue ("quarks") and the antifermions red ("antiquarks"), or vice versa, without changing any physics.

7.3 Solving the Gauss's law recurrence

Equation (67) is a simple first-order difference equation. Telescoping:

$$L_n = L_0 + \sum_{k=1}^{n}(L_k - L_{k-1}) = L_0 + \sum_{k=1}^{n} Q_k = \epsilon_0 + \sum_{k=1}^{n} Q_k. \tag{72}$$

Substituting the Pauli form (71):

$$\boxed{L_n = \epsilon_0 - \frac{1}{2}\sum_{k=1}^{n}\left[Z_k + (-1)^k\right].} \tag{73}$$

This expresses the electric field on link n as a linear combination of Z operators on qubits 1 through n, plus some constants. The gauge field is **gone**: the Hamiltonian is now purely in terms of Pauli operators on qubits. This is the magic of (1+1)D: there is no photon degree of freedom to track.

7.4 Interpretation of the background field ϵ_0

The boundary condition $L_0 = \epsilon_0$ introduces a free parameter. Physically, ϵ_0 is the electric field on an imaginary link to the left of site 1, and it represents *background* electric flux pointing through the entire system. In the continuum Schwinger model, this background is related to the topological *theta angle*:

$$\theta = 2\pi\epsilon_0 \pmod{2\pi}. \tag{74}$$

The theta angle labels inequivalent superselection sectors of the model. At $\theta = 0$ the vacuum is invariant under charge conjugation; at $\theta = \pi$ it is not, and for certain values of m/g the $\theta = \pi$ vacuum undergoes a second-order phase transition (the Coleman phase transition). This is interesting physics we can expose to the player via a slider: dragging ϵ_0 from 0 to 1 drags θ from 0 to 2π, and the vacuum rearranges in visible ways.

For the purposes of implementation, ϵ_0 is just a real-valued parameter that appears in (73). At $\epsilon_0 = 0$ the background vanishes and the vacuum is the standard one; at nonzero ϵ_0 the vacuum has a uniform background field whose energy shifts the whole spectrum.

8 The Final Qubit Hamiltonian

We now assemble everything into the final form, which is what the code will implement.

$$H = \underbrace{\frac{1}{4a}\sum_{n=1}^{N-1}(X_nX_{n+1} + Y_nY_{n+1})}_{H_{\text{hop}}} + \underbrace{\frac{m}{2}\sum_{n=1}^{N}(-1)^n Z_n}_{H_{\text{mass}}} + \underbrace{\frac{ag^2}{2}\sum_{n=1}^{N-1} L_n^2}_{H_{\text{el}}} \tag{75}$$

with

$$L_n = \epsilon_0 - \frac{1}{2}\sum_{k=1}^{n}\left[Z_k + (-1)^k\right]. \tag{73}$$

This Hamiltonian acts on an N-qubit Hilbert space of dimension 2^N. It is Hermitian (every term is manifestly so). It contains only X, Y, and Z Pauli operators, so it can be entered directly into Qiskit's `SparsePauliOp` class. It has no gauge-field degrees of freedom: they have been eliminated. It has no fermion operators: they have been Jordan–Wigner-transformed.

8.1 Term counting

How many Pauli strings appear in H?

- **Hopping**: $2(N-1)$ two-qubit Pauli strings (XX and YY on each of $N-1$ links).
- **Mass**: N single-qubit Z strings.
- **Electric**: when we expand L_n^2, each L_n is a linear combination of a constant and up to n single-qubit Z operators. Squaring produces identity terms (from constant^2), single-Z terms (from constant$\times Z$), and ZZ terms (from $Z \times Z$). Summing over n from 1 to $N-1$, the total number of distinct Pauli strings in H_{el} is $O(N^2)$, but after collecting like terms, the count grows as $O(N^2)$.

In total the Hamiltonian has $O(N^2)$ distinct Pauli terms. For $N = 6$ this is $\sim$ 30–50 terms, well within Qiskit's capacity and a hardware-friendly size for IonQ.

8.2 Expanding the electric energy

For the sake of clarity (and because the coders need to know how L_n^2 decomposes), let us expand L_n^2 explicitly. Define

$$c_n \equiv \epsilon_0 - \frac{1}{2}\sum_{k=1}^{n}(-1)^k, \qquad \mathcal{Z}_n \equiv \frac{1}{2}\sum_{k=1}^{n} Z_k, \tag{76}$$

so that $L_n = c_n - \mathcal{Z}_n$ (note: c_n is just a real number depending on n and ϵ_0; $\mathcal{Z}_n$ is an operator). Then

$$L_n^2 = c_n^2 - 2c_n\mathcal{Z}_n + \mathcal{Z}_n^2. \tag{77}$$

Expanding $\mathcal{Z}_n^2$:

$$\mathcal{Z}_n^2 = \frac{1}{4}\sum_{k=1}^{n}\sum_{\ell=1}^{n} Z_k Z_\ell = \frac{1}{4}\left[\sum_{k=1}^{n} Z_k^2 + 2\sum_{1\le k<\ell\le n} Z_k Z_\ell\right] = \frac{n}{4}I + \frac{1}{2}\sum_{1\le k<\ell\le n} Z_k Z_\ell, \tag{78}$$

where we used $Z_k^2 = I$. Therefore

$$L_n^2 = \left(c_n^2 + \frac{n}{4}\right) I - c_n \sum_{k=1}^{n} Z_k + \frac{1}{2}\sum_{1\le k<\ell\le n} Z_k Z_\ell. \tag{79}$$

Summing over n and multiplying by $ag^2/2$ gives H_{el} as a linear combination of I, single-Z, and ZZ terms.

The coders do not need to simplify this by hand. They should construct each L_n as a `SparsePauliOp` via (73), square it using the built-in `SparsePauliOp` multiplication, sum over n, and multiply by the prefactor. Qiskit will handle all the algebra and collect like terms automatically. This is why we prefer `SparsePauliOp`: the whole point of building the Hamiltonian symbolically is that you do not have to trust your own algebra. See the pseudocode in Section 15.

8.3 Sanity check: the $N = 2$ case

Let us work out the Hamiltonian explicitly for $N = 2$ with parameters $a = 1$, m, g, ϵ_0. This is small enough to compute by hand and invaluable as a sanity check for the code.

Sites: $n = 1$ (odd) and $n = 2$ (even). Links: $n = 1$ (the only link).

Hopping:

$$H_{\text{hop}} = \frac{1}{4}(X_1X_2 + Y_1Y_2). \tag{80}$$

Mass:

$$H_{\text{mass}} = \frac{m}{2}\left[(-1)^1 Z_1 + (-1)^2 Z_2\right] = \frac{m}{2}(-Z_1 + Z_2). \tag{81}$$

Electric field:

$$L_1 = \epsilon_0 - \frac{1}{2}[Z_1 + (-1)^1] = \epsilon_0 - \frac{Z_1 - 1}{2} = \left(\epsilon_0 + \tfrac{1}{2}\right) - \frac{Z_1}{2}. \tag{82}$$

Squaring:

$$L_1^2 = \left(\epsilon_0 + \tfrac{1}{2}\right)^2 - \left(\epsilon_0 + \tfrac{1}{2}\right) Z_1 + \frac{Z_1^2}{4} = \left[\left(\epsilon_0 + \tfrac{1}{2}\right)^2 + \tfrac{1}{4}\right] I - \left(\epsilon_0 + \tfrac{1}{2}\right) Z_1. \tag{83}$$

So

$$H_{\text{el}} = \frac{g^2}{2} L_1^2 = \frac{g^2}{2}\left[\left(\epsilon_0 + \tfrac{1}{2}\right)^2 + \tfrac{1}{4}\right] I - \frac{g^2}{2}\left(\epsilon_0 + \tfrac{1}{2}\right) Z_1. \tag{84}$$

Total Hamiltonian (at $\epsilon_0 = 0$):

$$H_{N=2} = \frac{1}{4}(X_1X_2 + Y_1Y_2) + \frac{m}{2}(-Z_1 + Z_2) + \frac{g^2}{2}\left[\tfrac{1}{2}I - \tfrac{1}{2}Z_1\right]. \tag{85}$$

Simplifying:

$$H_{N=2} = \frac{1}{4}(X_1X_2 + Y_1Y_2) - \left(\frac{m}{2} + \frac{g^2}{4}\right) Z_1 + \frac{m}{2} Z_2 + \frac{g^2}{4} I. \tag{86}$$

This is a 4×4 Hermitian matrix in the basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. The student is encouraged to build this matrix by hand, diagonalize it (e.g., with `numpy.linalg.eigh`), and verify that the Qiskit-built Hamiltonian agrees. If it does not, there is a sign or bit-ordering bug somewhere, and this is the right place to find it.

Vacuum energy for $N = 2$, $\epsilon_0 = 0$: from our analysis above, $|\text{vac}\rangle = |01\rangle$ (site 1 in $|0\rangle$, site 2 in $|1\rangle$). In this state:

- $\langle X_1X_2\rangle = 0$, $\langle Y_1Y_2\rangle = 0$ (both off-diagonal).
- $\langle Z_1\rangle = +1$, $\langle Z_2\rangle = -1$.
- H_{mass} contribution: $(m/2)(-1 + (-1)) = -m$.
- H_{el} contribution: $(g^2/2) \cdot \frac{1}{2} \cdot (1-1)^2 + \ldots$ wait, better to use the formula directly: $L_1 |01\rangle = (1/2)(1 - Z_1)|01\rangle = (1/2)(1-1)|01\rangle = 0$. So $L_1^2 |01\rangle = 0$ and the electric energy is zero in the vacuum (as expected: no charges, no field, no energy, at $\epsilon_0 = 0$).

Total vacuum energy: $-m$. The formula $-mN/2 = -m$ checks out for $N = 2$.

9 Observables for the Frontend

The game displays quantum-mechanical expectation values in real time. These are:

9.1 Electric field on each link

From (73),

$$\langle L_n \rangle = \epsilon_0 - \frac{1}{2} \sum_{k=1}^{n} \left[\langle Z_k \rangle + (-1)^k \right]. \tag{87}$$

Because L_n is a linear combination of Z operators, a *single* computational-basis measurement of all N qubits gives all N $\langle Z_k \rangle$ values (as shot averages), from which all $N-1$ link fields follow algebraically. One circuit run per frame. Use this to color the field bars between sites.

9.2 Particle number and charge at each site

Site occupation:

$$\langle \phi_n^\dagger \phi_n \rangle = \frac{1 - \langle Z_n \rangle}{2}. \tag{88}$$

Charge:

$$\langle Q_n \rangle = -\frac{\langle Z_n \rangle + (-1)^n}{2}. \tag{89}$$

Both are derived from the same Z-basis measurement as the electric field, and require no additional circuits.

9.3 Total energy

This is $\langle H \rangle$. Unlike the previous observables, H contains X and Y Pauli strings (from the hopping), so we need multiple measurement bases: Z-basis shots give us $\langle Z \rangle$-type and $\langle ZZ \rangle$-type expectation values; X-basis shots give us $\langle XX \rangle$ on each link; Y-basis shots give us $\langle YY \rangle$ on each link. The **Qiskit `Estimator` primitive** handles all of this automatically: you pass it a `SparsePauliOp` (our H) and a state (our circuit), and it returns $\langle H \rangle$, doing the basis grouping and weighted-sum internally.

Do not write your own Pauli measurement grouping. It is a correct, efficient, and well-tested routine in Qiskit's primitives. Use it.

9.4 Chiral condensate (bonus observable)

The chiral condensate $\langle \bar{\psi}\psi \rangle$ is the conventional order parameter for chiral symmetry in the Schwinger model. In the staggered formulation it becomes

$$\langle \bar{\psi}\psi \rangle \to \frac{1}{N} \sum_{n=1}^{N} (-1)^n \langle \phi_n^\dagger \phi_n \rangle = -\frac{1}{2N} \sum_{n=1}^{N} (-1)^n \langle Z_n \rangle \tag{90}$$

(dropping a constant for even N). This is proportional to $\langle H_{\text{mass}} \rangle / m$. If we already have the mass energy, this observable is free. Display it on the dashboard as an extra "order parameter" gauge for the physics-minded judges to notice.

10 Initial State Preparation

The backend must be able to prepare several specific initial states, corresponding to different game configurations.

10.1 The bare vacuum

The state $|\text{vac}\rangle = |0101\ldots01\rangle$ is prepared by starting from Qiskit's default $|00\ldots0\rangle$ and applying an X gate to every even-numbered site (using our 1-indexed site labels, i.e., $n = 2, 4, \ldots, N$; in Python 0-indexed, this is $n = 1, 3, \ldots, N-1$). That is the entire preparation: $N/2$ single-qubit gates, no entanglement.

10.2 A single fermion-antifermion pair

To represent a quark-antiquark pair with the quark at an odd site p and the antiquark at an even site q (with $p < q$, say):

1. Prepare $|\text{vac}\rangle$.
2. Apply X_p (this flips qubit p from $|0\rangle$ to $|1\rangle$, creating a fermion at site p).
3. Apply X_q (this flips qubit q from $|1\rangle$ to $|0\rangle$, creating an antifermion at site q).

Total cost: $(N/2) + 2$ single-qubit X gates from the all-zero start. The resulting state has:

- $\langle Q_p \rangle = +1$ (fermion at odd site p),
- $\langle Q_q \rangle = -1$ (antifermion at even site q),
- $\langle L_n \rangle = \epsilon_0 + 1$ for $p \leq n < q$, and $\langle L_n \rangle = \epsilon_0$ elsewhere. This nonzero electric field between the charges is the "string" that the player will see in the game.

10.3 Why "drag" is a re-initialization, not a gate

When the player drags a particle from site p to site p' in the game, the backend does *not* apply a unitary operation to the existing state. Instead, it discards the current state entirely, prepares a new product state with the pair at (p', q), and lets the player decide whether to evolve it. There is no "move charge" gate in our circuit library, and any attempt to invent one will turn into a mess, because moving a charge nonlocally changes the electric field everywhere along the path, and the only thing that enforces this is Gauss's law, which only constrains states not operators.

This is a critical point and easy to miss. It is listed in the "do not do" section (Section 15.6) as well.

11 Time Evolution and Trotterization

When the player clicks "evolve," the game advances the wavefunction by a small time step Δt under the Hamiltonian H. This means applying the unitary $U(\Delta t) = e^{-iH\Delta t}$. The challenge is that H is a sum of non-commuting pieces:

$$[H_{\text{hop}}, H_{\text{mass}}] \neq 0, \qquad [H_{\text{hop}}, H_{\text{el}}] \neq 0, \qquad [H_{\text{mass}}, H_{\text{el}}] = 0. \tag{91}$$

(The mass and electric terms commute because both are diagonal in the Z basis.) Because of the non-commutation, we cannot implement $e^{-iH\Delta t}$ exactly as a product of exponentials of each piece; we need the Trotter (Lie–Trotter–Suzuki) product formula to approximate it.

11.1 First-order Trotter

The simplest approximation is

$$e^{-iH\Delta t} \approx e^{-iH_{\text{hop}}\Delta t}\, e^{-iH_{\text{mass}}\Delta t}\, e^{-iH_{\text{el}}\Delta t}, \tag{92}$$

with error $O(\Delta t^2)$ per step, coming from the non-commutation of the three terms. Each factor on the right is a product of commuting exponentials that can be implemented as gates:

- $e^{-iH_{\text{mass}}\Delta t}$: a product of single-qubit R_z gates, one per site. Specifically, $R_z(\theta_n)$ where $\theta_n = m(-1)^n\Delta t$. (The factor-of-two convention: in Qiskit, $R_z(\theta) = \exp(-i\theta Z/2)$, so if the Hamiltonian term is $(m/2)(-1)^n Z_n$, the evolution is $\exp(-i(m/2)(-1)^n Z_n\Delta t)$, matching $R_z(m(-1)^n\Delta t)$.)
- $e^{-iH_{\text{el}}\Delta t}$: a product of single-qubit R_z gates and two-qubit R_{zz} gates (where $R_{zz}(\theta) = \exp(-i\theta Z \otimes Z/2)$), because H_{el} is a sum of Z, ZZ, and identity terms. All these terms are mutually commuting, so their exponentials factor exactly.
- $e^{-iH_{\text{hop}}\Delta t}$: this one is the most interesting. Each link contributes $(X_nX_{n+1} + Y_nY_{n+1})/(4a)$ to H_{hop}. On a single link, X_nX_{n+1} and Y_nY_{n+1} commute (easy to check: $XXYY = X \cdot Y \cdot X \cdot Y = (iZ)(iZ) = -I$, and similarly $YYXX = -I$), so the single-link exponential factors as $\exp(-i(\Delta t/(4a))X_nX_{n+1})\exp(-i(\Delta t/(4a))Y_nY_{n+1})$. However, exponentials on *different* links do not commute. The workaround: split the links into "odd" ($n = 1, 3, 5, \ldots$) and "even" ($n = 2, 4, 6, \ldots$) classes. Within each class, links act on disjoint qubits, so their exponentials commute, and we can apply them in parallel. The cost is that we introduce another level of Trotter error by alternating the two classes.

11.2 Second-order Trotter (Strang splitting)

For better accuracy at the same computational cost per time unit, we use second-order Suzuki–Trotter:

$$e^{-iH\Delta t} \approx e^{-iH_{\text{el}}\Delta t/2}\, e^{-iH_{\text{mass}}\Delta t/2}\, e^{-iH_{\text{hop}}\Delta t}\, e^{-iH_{\text{mass}}\Delta t/2}\, e^{-iH_{\text{el}}\Delta t/2}, \tag{93}$$

with error $O(\Delta t^3)$ per step. Because the error is one power smaller, you can afford a larger Δt (or equivalently, fewer total steps for a given evolution time), more than making up for the doubled per-step gate count. **Always use second-order Trotter unless you have a reason not to.**

11.3 Qiskit implementation

Do not implement Trotterization by hand. Qiskit provides `PauliEvolutionGate`, which takes a `SparsePauliOp` (our H) and a time Δt, and emits a circuit implementing $e^{-iH\Delta t}$ to a user-specified Trotter order. Code sketch:

```
from qiskit.circuit.library import PauliEvolutionGate
from qiskit.synthesis import SuzukiTrotter

H_op = build_hamiltonian(N, m, g, a, eps0)  # SparsePauliOp
evo_gate = PauliEvolutionGate(
    H_op,
    time=dt,
    synthesis=SuzukiTrotter(order=2, reps=1)
)
circuit.append(evo_gate, range(N))
```

To evolve by k steps, either append this gate k times to the circuit, or set `time=k*dt`. The first approach costs more gates but is exact in each step; the second approach costs fewer gates but accumulates Trotter error over the whole interval. For the game, prefer appending one gate per click.

11.4 Step size and accuracy

Guidelines for Δt:

- Too small: nothing visible changes between frames; the game feels laggy and unresponsive.
- Too large: Trotter error dominates, energy is not conserved, and the visible dynamics look wrong.
- Sweet spot: Δt in the range 0.1 to 0.3 in lattice units (i.e., multiples of $a = 1$).

Tuning procedure: Pick an initial state, evolve it for 20 steps at $\Delta t = 0.2$, and measure the energy after each step. If the energy drifts by more than a few percent over 20 steps, halve Δt or increase `SuzukiTrotter reps` from 1 to 2. If it drifts by less than 1%, you are good.

Important reality check: Trotter error is *not* the same as quantum measurement noise. The simulator gives you the exact wavefunction, and Trotter error shows up as a deterministic drift of observables. On real hardware, you additionally get shot noise and gate noise on top of Trotter error. For the game on the simulator, Trotter error is the only error, and it is controllable.

12 Variational Quantum Eigensolver: Level 4

The secret sauce of our demo, from the sponsor's perspective, is level 4, where the player unknowingly performs the classical optimizer's role in a hybrid quantum–classical VQE algorithm.

12.1 VQE in three sentences

The variational quantum eigensolver approximates the ground state of a Hamiltonian H by: (1) preparing a parameterized quantum state $|\psi(\vec{\theta})\rangle = U(\vec{\theta})\,|\text{vac}\rangle$; (2) measuring the energy $\langle H\rangle = \langle\psi(\vec{\theta})|H|\psi(\vec{\theta})\rangle$; (3) classically updating $\vec{\theta}$ to try to minimize $\langle H\rangle$, and looping. The minimum achievable energy is an upper bound on the true ground state energy (by the variational principle), and it is exact if the ansatz is expressive enough to cover the true ground state. In practice the ansatz is never perfect, but it is often very good.

12.2 The hardware-efficient ansatz

We use a *hardware-efficient ansatz* with 2 layers. The structure is:

1. **Initial state:** prepare $|\text{vac}\rangle = |0101\ldots01\rangle$ with X gates on even sites.
2. **Layer 1:**
   (a) Apply $R_y(\theta_n^{(1)})$ to every qubit, $n = 1, \ldots, N$. (This introduces N parameters.)
   (b) Apply CNOTs in a linear chain: CNOT(1→2), CNOT(2→3), …, CNOT(N−1→N).
3. **Layer 2:** same as layer 1, with new parameters $\theta_n^{(2)}$.
4. **Final rotations:** apply $R_y(\theta_n^{(3)})$ to every qubit.

Total parameters: $3N$. For $N = 6$, that is 18 parameters.

12.3 Exposing the right parameters to the player

18 sliders is too many for a game. We will expose only 4 sliders; the remaining 14 parameters are fixed at values determined by a classical pre-optimization (run once, before the hackathon or on

Friday night). The procedure for choosing *which* 4:

1. Classically run VQE on the $N = 6$ Schwinger Hamiltonian to convergence using `scipy.optimize.minimize(m` or similar.
2. Record the converged parameter values $\vec{\theta}^*$.
3. Compute the sensitivity $|\partial\langle H\rangle/\partial\theta_i|$ at $\vec{\theta}^*$ for each parameter i, using finite differences.
4. Pick the 4 parameters with the highest sensitivity.

These are the 4 parameters that matter most, in the sense that changing them shifts the energy the most. Exposing them as sliders gives the player the most impact per drag.

Note: this classical pre-optimization is a one-time cost, done before the demo. During the game, the player is not running the classical optimizer — the player *is* the classical optimizer, manually tuning the exposed parameters to find the minimum. The "target" energy is displayed as a line on a thermometer: the player can see how close they are to the exact ground state (found via exact diagonalization, Section 13) and to the optimal VQE energy (found via classical optimization).

12.4 The pitch line

"You are the VQE optimizer. A classical computer usually does this job, tweaking parameters to minimize the energy of a quantum state. Tonight, you are that computer. Can you beat the algorithm?" Then after the player converges (or gives up), the demo plays back a recording of the classical optimizer's trajectory through parameter space, showing whether the player was faster or slower. Judges will eat this up.

13 Exact Diagonalization: The Ground Truth

Because N is small ($N \leq 8$), we can build the full $2^N \times 2^N$ Hamiltonian matrix and diagonalize it classically. This is our *ground truth*: every expectation value that our quantum circuits produce should agree with it on the simulator, to within Trotter error (for dynamics) or to within machine precision (for state preparation and static observables).

The entire reference code is 10 lines:

```
import numpy as np
from qiskit.quantum_info import SparsePauliOp, Statevector

H_op = build_hamiltonian(N=6, m=0.5, g=1.0, a=1.0, eps0=0.0)
H_matrix = H_op.to_matrix()  # 2^N x 2^N numpy array

# Exact diagonalization:
eigvals, eigvecs = np.linalg.eigh(H_matrix)
E0_exact = eigvals[0]
psi0 = eigvecs[:, 0]

# Exact expectation values:
def exact_expect(op_matrix, state_vec):
    return (state_vec.conj() @ op_matrix @ state_vec).real
```

Use this to verify the Qiskit code path:

- Build H two ways: via `SparsePauliOp` + `to_matrix()`, and via hand-coded matrix multiplication for $N = 2$. They should agree to machine precision.

- Compute $\langle H \rangle$ in $|\text{vac}\rangle$ two ways: via `Statevector(prepare_vacuum(N)).expectation_value(H_op)`, and via the exact code above. They should agree.
- Same for $\langle Z_n \rangle$, $\langle L_n \rangle$, etc.

If any comparison fails, there is a bug (bit ordering, sign convention, or index origin), and this is the time to find it, not at 2am on Sunday.

14 Recommended Parameter Values

Here are the default parameters for the game. Do not change these without running a full regression test.

Parameter	**Default**	**Rationale**
N	6 (static), 8 (dynamic)	Big enough to show non-trivial structure; small enough to simulate in real time.
a	1	Lattice spacing set to 1 by convention. All energies are in units of $1/a$.
m	0.5	Intermediate between light ($m \ll 1/a$) and heavy ($m \gg 1/a$). Gives visible confinement without suppressing hopping.
g	1.0	Order-unity coupling. Stronger coupling increases the confinement scale (makes the string "stiffer").
ϵ_0	0 (slider: 0 to 1)	Background field/theta angle. Slider is the "theta" knob in the game.
Δt	0.2	Second-order Trotter step. Small enough for accuracy, large enough to feel responsive.
Trotter order	2	Second-order Suzuki.
Trotter reps/step	1	Bump to 2 if energy drift exceeds 2% over 20 steps.
VQE ansatz layers	2	Enough to express ground state to $\sim 1\%$ for $N = 6$ Schwinger.
VQE sliders exposed	4	Playable; see Section 12.

Dimensional consistency check. Each term in the Hamiltonian (75) has dimensions of energy $= 1/a$:

- Hopping: coefficient $1/(4a)$, which has units of $1/a$. ✓
- Mass: coefficient $m/2$, and m has units of energy $= 1/a$. ✓
- Electric: coefficient $ag^2/2$. In (1+1)D, g^2 has units of $1/a^2$ (see discussion after (41)), so ag^2 has units of $1/a$. ✓

All three pieces are in the same unit, so we can add them without unit confusion.

15 Implementation Checklist and Game-Action Mapping

This section is the literal handoff to the coders. Everything they need to implement is spelled out here, in order.

15.1 The `SchwingerGameState` class

The core backend object. It encapsulates all the quantum state and offers a clean API to the game frontend.

```
class SchwingerGameState:
    def __init__(self, N: int, m: float, g: float, a: float = 1.0,
                 eps0: float = 0.0, backend: str = "simulator"):
        """
        N: number of lattice sites (even, 4-8)
        m, g, a: physical parameters (see Section 14)
        eps0: background electric field (slider)
        backend: 'simulator' or 'ionq'
        """
        self.N = N
        self.m = m
        self.g = g
        self.a = a
        self.eps0 = eps0
        self.backend = backend
        self.H_op = self._build_hamiltonian()  # SparsePauliOp
        self.H_matrix = self.H_op.to_matrix()  # for classical checks
        self.circuit = self._prepare_vacuum()  # QuantumCircuit

    def _build_hamiltonian(self) -> SparsePauliOp:
        """Build H according to Eq. (eq:H-final) as a SparsePauliOp."""
        ...

    def _prepare_vacuum(self) -> QuantumCircuit:
        """Return a circuit preparing |0101...01>."""
        ...

    def reset_to_vacuum(self):
        """Discard current circuit, rebuild vacuum."""
        self.circuit = self._prepare_vacuum()

    def place_pair(self, p: int, q: int):
        """
        Reset to vacuum, then create a fermion at odd site p and
        an antifermion at even site q. p and q are 1-indexed.
        """
        self.reset_to_vacuum()
        self.circuit.x(p - 1)  # 0-indexed conversion
        self.circuit.x(q - 1)

    def evolve(self, dt: float, steps: int = 1):
        """Append second-order Trotter steps to the circuit."""
        from qiskit.circuit.library import PauliEvolutionGate
        from qiskit.synthesis import SuzukiTrotter
        for _ in range(steps):
            gate = PauliEvolutionGate(
                self.H_op, time=dt,
                synthesis=SuzukiTrotter(order=2, reps=1)
            )
            self.circuit.append(gate, range(self.N))

    def set_coupling(self, g_new: float):
        self.g = g_new
        self.H_op = self._build_hamiltonian()
```

```
        self.H_matrix = self.H_op.to_matrix()
        # Do NOT touch self.circuit: the state is unchanged,
        # only the Hamiltonian governing its evolution has changed.

    def set_epsilon(self, eps_new: float):
        self.eps0 = eps_new
        self.H_op = self._build_hamiltonian()
        self.H_matrix = self.H_op.to_matrix()

    def vqe_layer(self, thetas: list[float]):
        """Apply the VQE ansatz (Section 13) with given parameters."""
        ...

    def measure_observables(self) -> dict:
        """
        Return a dict with keys:
          'Z': list of <Z_n>, n=1..N
          'L': list of <L_n>, n=1..N-1
          'Q': list of <Q_n>, n=1..N
          'energy': <H>
        """
        ...
```

15.2 Building the Hamiltonian

```
from qiskit.quantum_info import SparsePauliOp

def build_hamiltonian(N: int, m: float, g: float, a: float,
                      eps0: float) -> SparsePauliOp:
    """
    Build the Schwinger model Hamiltonian as a SparsePauliOp.

    CRITICAL WARNINGS:
    - Qiskit uses LITTLE-ENDIAN Pauli strings. 'XXII' means X on
      qubits 2,3 (the RIGHTMOST bits), not qubits 0,1. ALWAYS REVERSE
      your Pauli list before joining into a string.
    - Math sites are 1-indexed (n=1..N); Python is 0-indexed (n=0..N-1).
      The sign on the mass term is (-1)^n in math, which with 0-indexed
      Python sites becomes (-1)^(n+1). Test this with N=2 and compare
      against the explicit formula in Section 8.3.
    """
    terms = []

    # --- Hopping: (1/(4a)) * sum_{n=1}^{N-1} (X_n X_{n+1} + Y_n Y_{n+1})
    for n in range(N - 1):  # 0-indexed link: connects qubits n, n+1
        for P in ('X', 'Y'):
            pauli = ['I'] * N
            pauli[n] = P
            pauli[n + 1] = P
            # Reverse for Qiskit little-endian:
            pauli_str = ''.join(pauli[::-1])
            terms.append((pauli_str, 1.0 / (4.0 * a)))

    # --- Mass: (m/2) * sum_{n=1}^{N} (-1)^n Z_n
    for n_math in range(1, N + 1):  # math 1-indexed
        n_py = n_math - 1
        sign = (-1) ** n_math
        pauli = ['I'] * N
```

```
        pauli[n_py] = 'Z'
        pauli_str = ''.join(pauli[::-1])
        terms.append((pauli_str, sign * m / 2.0))

    H = SparsePauliOp.from_list(terms)

    # --- Electric: (a g^2 / 2) * sum_{n=1}^{N-1} L_n^2
    H_el = _build_electric_energy(N, g, a, eps0)
    H = (H + H_el).simplify()

    return H

def _build_L_n(n_math: int, N: int, eps0: float) -> SparsePauliOp:
    """Build L_n = eps0 - (1/2) sum_{k=1}^{n} [Z_k + (-1)^k] as a SparsePauliOp.
        """
    # Start with the constant part.
    const = eps0
    for k in range(1, n_math + 1):
        const -= 0.5 * ((-1) ** k)
    terms = [('I' * N, const)]
    # Add the -Z_k/2 pieces.
    for k_math in range(1, n_math + 1):
        k_py = k_math - 1
        pauli = ['I'] * N
        pauli[k_py] = 'Z'
        pauli_str = ''.join(pauli[::-1])
        terms.append((pauli_str, -0.5))
    return SparsePauliOp.from_list(terms).simplify()

def _build_electric_energy(N, g, a, eps0) -> SparsePauliOp:
    """Build (a g^2 / 2) * sum_n L_n^2."""
    prefactor = a * g ** 2 / 2.0
    H_el = None
    for n_math in range(1, N):  # links 1..N-1
        L_n = _build_L_n(n_math, N, eps0)
        L_n_sq = (L_n @ L_n).simplify()  # or L_n.compose(L_n)
        contribution = prefactor * L_n_sq
        H_el = contribution if H_el is None else (H_el + contribution)
    return H_el.simplify()
```

15.3 Game action → quantum operation table

This is the complete mapping. Every user interaction in the game must correspond to exactly one row of this table. If the frontend has an action not listed here, ask the physics lead what it should do quantum-mechanically; do not invent.

Game action	Quantum operation
Player starts a level	`state.reset_to_vacuum();` `state.place_pair(p_0, q_0)` with the level's initial (p_0, q_0)
Player drags the fermion to a new site p'	`state.place_pair(p', q)` [re-init, not a gate]
Player drags the antifermion to a new site q'	`state.place_pair(p, q')`
Player clicks "evolve one step"	`state.evolve(dt=0.2, steps=1)`
Player drags the coupling slider to g'	`state.set_coupling(g')` [rebuilds H, does not touch state]
Player drags the θ slider to θ'	`state.set_epsilon(theta' / (2*pi))` [rebuilds H]
Player drags the mass slider to m'	(add analogous `set_mass` method)
Player resets the level	`state.reset_to_vacuum();` `state.place_pair(p_0, q_0)`
Player adjusts a VQE slider (level 4 only)	`state.reset_to_vacuum();` `state.vqe_layer(new_thetas)`
Every frame (after any action)	`obs = state.measure_observables(); push to frontend`

15.4 Caching for responsiveness

The backend should cache results: a tuple `(circuit_hash, parameter_hash)` → dict of observables. A slider drag that revisits a previously-computed parameter value should return the cached result, not re-run the simulator. Slider gestures often sweep through dozens of values per second; running the simulator for each is wasteful.

15.5 The one dramatic hardware run

Do not run every frame on real IonQ hardware. The queue and the cost are prohibitive. Instead:

1. All gameplay runs on the classical simulator (`Statevector` or `AerSimulator`).
2. On Saturday night, pick *one* representative circuit (e.g., the "String Breaker" level's signature state at its most interesting time slice) and submit it to IonQ via the cloud.
3. Wait for the results on Sunday morning.
4. During the demo, there is a special "Run on real quantum hardware" button that, when pressed, displays the pre-computed hardware results side by side with the simulator results. Note the discrepancy (hardware has gate noise) and explain it to the judges: "This is the real deal — the slight difference is actual quantum noise."

This gives us the drama of real hardware without the risk of a queue disaster.

15.6 Things the coders should not do

- **Do not implement Kogut–Susskind with explicit gauge qubits.** We eliminated the gauge field. Tutorials that keep gauge link qubits exist, and they are wrong for our purposes. Ignore them.

- **Do not hand-code Trotter decompositions.** Use `PauliEvolutionGate` with `SuzukiTrotter`.
- **Do not hand-code Pauli measurement grouping for the energy.** Use the `Estimator` primitive.
- **Do not set $N > 8$.** Simulation time grows exponentially, and the game loses responsiveness.
- **Do not implement error mitigation** (zero-noise extrapolation, readout calibration, probabilistic error cancellation). Interesting but a rabbit hole; not needed for the demo.
- **Do not run every frame on real IonQ hardware.** Simulator by default; one dramatic hardware run for the demo.
- **Do not implement drag as a gate.** It is a re-initialization. See Section 10.
- **Do not change sign or index conventions mid-project.** If you find a sign issue, fix it globally, re-run exact diagonalization to verify, and push one commit.
- **Do not trust the code before it matches the $N = 2$ hand calculation** in Section 8.3. This is the gatekeeper test.
- **Do not try to beat IonQ's SDK** by writing your own quantum backend. Use `qiskit_ionq` or the official provider. Our job is physics and game, not cloud plumbing.

15.7 The recommended build order

A suggested timeline for the 36 hours:

1. **Hour 0–2:** Set up the repo, install Qiskit, get `build_hamiltonian` working for $N = 2$ and verify against the $N = 2$ hand calculation.
2. **Hour 2–4:** Extend to $N = 4, 6$. Build exact-diagonalization cross-check. Make sure `Estimator` returns the same ground-state energy as `numpy.linalg.eigh`.
3. **Hour 4–6:** State preparation and observables. Confirm that $|\text{vac}\rangle$ has the expected expectation values.
4. **Hour 6–10:** Time evolution. Apply a single Trotter step to a fermion-antifermion pair state; watch the electric field change. Verify energy conservation.
5. **Hour 10–14:** VQE ansatz. Classically optimize; record the converged $\vec{\theta}^*$ and sensitivity ranking.
6. **Hour 14–20:** FastAPI backend wrapping the `SchwingerGameState`, with caching. JSON endpoints for each game action.
7. **Hour 20–28:** Frontend integration, game levels, art pass.
8. **Hour 28–32:** Submit IonQ hardware run. Record results.
9. **Hour 32–36:** Polish, demo rehearsal, debugging.

16 References and Further Reading

1. J. Kogut and L. Susskind, "Hamiltonian formulation of Wilson's lattice gauge theories," Phys. Rev. D **11**, 395 (1975). The original paper on Hamiltonian lattice gauge theory with staggered fermions.

2. T. Banks, L. Susskind, and J. Kogut, "Strong-coupling calculations of lattice gauge theories: $(1 + 1)$-dimensional exercises," Phys. Rev. D **13**, 1043 (1976). Where the lattice Schwinger model was worked out in detail.

3. E. A. Martinez et al., "Real-time dynamics of lattice gauge theories with a few-qubit quantum computer," Nature **534**, 516 (2016). The first experimental quantum simulation of the Schwinger model, on 4 trapped ions — very close to what we are doing, and an excellent visual reference for what the demo should look like.

4. M. C. Bañuls and K. Cichy, "Review on novel methods for lattice gauge theories," Rep. Prog. Phys. **83**, 024401 (2020), arXiv:1910.00257. A thorough modern review; read for a broader context.

5. A. F. Shaw, P. Lougovski, J. R. Stryker, and N. Wiebe, "Quantum Algorithms for Simulating the Lattice Schwinger Model," Quantum **4**, 306 (2020), arXiv:2002.11146. Careful resource estimates and algorithmic details; useful if we want to push toward larger N or more sophisticated Trotter schemes.

A Appendix: Anticommutation of Jordan–Wigner Operators

We verify that the JW-transformed operators (45) and (46) satisfy the fermionic anticommutation relations $\{\phi_m, \phi_n^\dagger\} = \delta_{mn}$ and $\{\phi_m, \phi_n\} = 0$. The case $m = n$ is immediate: the JW strings cancel, and $\{\sigma_n^+, \sigma_n^-\} = I$ (a direct computation from the definitions). We focus on $m < n$.

Define the string operator $S_n = \prod_{\ell=1}^{n-1}(-Z_\ell)$, so $\phi_n = S_n\sigma_n^+$ and $\phi_n^\dagger = S_n\sigma_n^-$. We use the on-site identities (easy to verify):

$$\sigma^+ Z = -\sigma^+, \quad Z\sigma^+ = \sigma^+, \quad \sigma^- Z = \sigma^-, \quad Z\sigma^- = -\sigma^-. \tag{94}$$

The sign of the anticommutator $\{\phi_m, \phi_n\}$ for $m < n$ reduces to how many Z's lie between sites m and n in the strings, and whether they pass through the raising/lowering operators at the endpoints. A careful bookkeeping gives that, for $m < n$, the cross-term contributes a factor of -1, so $\phi_m\phi_n = -\phi_n\phi_m$ and hence $\{\phi_m, \phi_n\} = 0$. Similarly, $\phi_m\phi_n^\dagger + \phi_n^\dagger\phi_m = \delta_{mn}$, with the Kronecker delta arising only from the $m = n$ case. A fully detailed walk-through of the bookkeeping is in the original Jordan–Wigner paper (1928) and in any quantum many-body textbook. For our purposes, what matters is that the identification (45)–(46) is mathematically correct, and therefore the Hamiltonian (75) is an exact transcription of the Kogut–Susskind Hamiltonian (41).

B Appendix: A Complete Pauli Decomposition for $N = 4$

As a check and a gift to the coders, we write out the full Pauli decomposition of H for $N = 4$, $a = 1$, $m = 0.5$, $g = 1$, $\epsilon_0 = 0$. This is a state space of dimension 16, small enough to diagonalize by hand (or with a short Python script) and compare to Qiskit.

Hopping:

$$H_{\text{hop}} = \frac{1}{4}\Big[X_1X_2 + Y_1Y_2 + X_2X_3 + Y_2Y_3 + X_3X_4 + Y_3Y_4\Big]. \tag{95}$$

Mass:

$$H_{\text{mass}} = \frac{0.5}{2}\Big[-Z_1 + Z_2 - Z_3 + Z_4\Big] = -0.25\,Z_1 + 0.25\,Z_2 - 0.25\,Z_3 + 0.25\,Z_4. \tag{96}$$

Electric (at $\epsilon_0 = 0$): we need L_n for $n = 1, 2, 3$.

$$L_1 = -\frac{1}{2}[Z_1 + (-1)^1] = -\frac{Z_1 - 1}{2} = \frac{1}{2} - \frac{Z_1}{2}, \tag{97}$$

$$L_2 = -\frac{1}{2}[Z_1 - 1 + Z_2 + 1] = -\frac{Z_1 + Z_2}{2}, \tag{98}$$

$$L_3 = -\frac{1}{2}[Z_1 - 1 + Z_2 + 1 + Z_3 - 1] = -\frac{Z_1 + Z_2 + Z_3 - 1}{2} = \frac{1}{2} - \frac{Z_1 + Z_2 + Z_3}{2}. \tag{99}$$

Squaring each:

$$L_1^2 = \left(\tfrac{1}{2} - \tfrac{Z_1}{2}\right)^2 = \tfrac{1}{4} - \tfrac{1}{2}Z_1 + \tfrac{1}{4}Z_1^2 = \tfrac{1}{2}I - \tfrac{1}{2}Z_1, \tag{100}$$

$$L_2^2 = \tfrac{1}{4}(Z_1 + Z_2)^2 = \tfrac{1}{4}(Z_1^2 + 2Z_1Z_2 + Z_2^2) = \tfrac{1}{2}I + \tfrac{1}{2}Z_1Z_2, \tag{101}$$

$$\begin{aligned} L_3^2 &= \left(\tfrac{1}{2} - \tfrac{Z_1+Z_2+Z_3}{2}\right)^2 \\ &= \tfrac{1}{4} - \tfrac{1}{2}(Z_1 + Z_2 + Z_3) + \tfrac{1}{4}(Z_1 + Z_2 + Z_3)^2 \\ &= \tfrac{1}{4}I - \tfrac{1}{2}(Z_1 + Z_2 + Z_3) + \tfrac{1}{4}(3I + 2Z_1Z_2 + 2Z_1Z_3 + 2Z_2Z_3) \\ &= I - \tfrac{1}{2}Z_1 - \tfrac{1}{2}Z_2 - \tfrac{1}{2}Z_3 + \tfrac{1}{2}Z_1Z_2 + \tfrac{1}{2}Z_1Z_3 + \tfrac{1}{2}Z_2Z_3. \end{aligned} \tag{102}$$

Summing ($(ag^2/2) = 1/2$):

$$\begin{aligned} H_{\text{el}} &= \tfrac{1}{2}\Big[L_1^2 + L_2^2 + L_3^2\Big] \\ &= \tfrac{1}{2}\Big[\tfrac{1}{2}I - \tfrac{1}{2}Z_1 + \tfrac{1}{2}I + \tfrac{1}{2}Z_1Z_2 + I - \tfrac{1}{2}Z_1 - \tfrac{1}{2}Z_2 - \tfrac{1}{2}Z_3 \\ &\qquad + \tfrac{1}{2}Z_1Z_2 + \tfrac{1}{2}Z_1Z_3 + \tfrac{1}{2}Z_2Z_3\Big] \\ &= I - \tfrac{1}{2}Z_1 - \tfrac{1}{4}Z_2 - \tfrac{1}{4}Z_3 + \tfrac{1}{2}Z_1Z_2 + \tfrac{1}{4}Z_1Z_3 + \tfrac{1}{4}Z_2Z_3. \end{aligned} \tag{103}$$

(Double-check arithmetic: $\frac{1}{2}[\frac{1}{2} + \frac{1}{2} + 1]I = I$; $\frac{1}{2}[-\frac{1}{2} - \frac{1}{2}]Z_1 = -\frac{1}{2}Z_1$; $\frac{1}{2}[-\frac{1}{2}]Z_2 = -\frac{1}{4}Z_2$; $\frac{1}{2}[-\frac{1}{2}]Z_3 = -\frac{1}{4}Z_3$; $\frac{1}{2}[\frac{1}{2} + \frac{1}{2}]Z_1Z_2 = \frac{1}{2}Z_1Z_2$; $\frac{1}{2} \cdot \frac{1}{2}\, Z_1Z_3 = \frac{1}{4}Z_1Z_3$; similarly Z_2Z_3.) ✓

Total H for $N = 4$, $a = m$ etc. as above:

$$\begin{aligned} H = \tfrac{1}{4}\Big[&X_1X_2 + Y_1Y_2 + X_2X_3 + Y_2Y_3 + X_3X_4 + Y_3Y_4\Big] \\ &+ (-0.25 - 0.5)Z_1 + (0.25 - 0.25)Z_2 + (-0.25 - 0.25)Z_3 + 0.25\, Z_4 \\ &+ \tfrac{1}{2}Z_1Z_2 + \tfrac{1}{4}Z_1Z_3 + \tfrac{1}{4}Z_2Z_3 + I. \end{aligned} \tag{104}$$

Collecting:

$$\begin{aligned} H_{N=4} = \tfrac{1}{4}(X_1X_2 + Y_1Y_2 + X_2X_3 + Y_2Y_3 + X_3X_4 + Y_3Y_4) \\ -\tfrac{3}{4}Z_1 + 0 \cdot Z_2 - \tfrac{1}{2}Z_3 + \tfrac{1}{4}Z_4 \\ +\tfrac{1}{2}Z_1Z_2 + \tfrac{1}{4}Z_1Z_3 + \tfrac{1}{4}Z_2Z_3 + I. \end{aligned} \tag{105}$$

This is the complete 16×16 Hamiltonian for $N = 4$ at the default parameters (modulo the identity shift which does not affect dynamics). Building this in Python and diagonalizing gives a specific ground-state energy. Qiskit's `SparsePauliOp`-based construction should give *exactly the same* matrix (up to numerical precision) and *exactly the same* ground-state energy. This is the acceptance test for the Hamiltonian builder.

End of specification. Questions to physics lead during hackathon.